

Dependency Injection in Spring framework

- Q In production grade environments, why is loose-coupling preferred?
- ⇒ We might need to change configurations in future, and hence want the application to be more adaptable to incorporate any changes.
- ⇒ With loose-coupling, we get independent testing, making application easier to test
- ⇒ We can introduce new functionalities w/o changing or modifying previous code
- ⇒ It reduces 'scattered object creation' and makes application scalable

Q What is Dependency Injection?

⇒ It's a design pattern, where an object receives it's dependence from outside, instead of creating them itself.



Constructor Injection

→ We couple the dependent class and the dependency injection through the constructor of the dependent class

```
class Application {
```

```
    (no Autowired)
```

```
    private final NotificationService notificationService;
```

Constructor Injection



```
    Application (NotificationService notificationService, ...) {
```

```
        this.notificationService = notificationService;
```

```
        // ...
```

```
    }
```

```
}
```

// Using Interfaces for Abstraction

```
public interface NotificationService {
```

```
    public void send (String message);
```

```
}
```

// Service classes which implement it

@Component

```
public class SmsNotificationService implements NotificationService {
```

```
    @Override
```

```
    public void send (String message) {
```

```
        |    System.out.println ("Sending SMS... " + message);
```

```
    }
```

```
}
```

@Component

```
public class EmailNotificationService implements NotificationService {  
  
    @Override  
    public void send (String message) {  
        System.out.println (" Sending email... " + message );  
    }  
}
```

Q How will springboot decide which bean to inject?

⇒ when multiple bean's are present, we can distinguish / dictate bean injection using these criteria's

1) Using @Primary annotation → if conflict is there, this annotated bean will take precedence

2) Using @Qualifier annotation

@Component

@Qualifier (" sms ")

public class SmsNotificationService ...

@Component

@Qualifier (" email ")

public class EmailNotificationService ...

And then mention appropriate qualifier during constructor injection

```
Application ( Qualifier ("email") NotificationService notificationService ... ) {  
    this.notificationService = notificationService;  
    // ...  
}
```

3) Using @Conditional & @Property

→ For this, we have to define key-value pair in either the application's properties, or in the application's environment

→ environment's properties take precedence over applications' properties

So, `notification.type = sms` OR `NOTIFICATION_TYPE = sms`
(prop) (env)

On the service class, mention

@ Component

@Conditionals On Property (name = "notification.type", havingValue = "sms")

```
public class SmsNotificationService ...
```

@Component

@Conditionals On Property (name = "notification.type", havingValue = "email")

```
public class EmailNotificationService ...
```

Q What if we want multiple bears w/o conflict?

⇒ We can @Autowired all the dependencies into a Map interface using HashMap object.

@ Autowired

```
Map<String, NotificationService> notifServiceMap = new HashMap<>();
```

// Usage

```
for ( var notificationService : notifServiceMap.entrySet() ) {  
    System.out.println ( notificationService.getKey());  
    notificationService.getValue().send();  
    notificationService.getValue()  
    actual bean object  
}
```